MNR UNIVERSITY

School of Engineering & Technology (SOET)
Department of Computer Science and Engineering

MINI PROJECT REPORT

GREEN CAMPUS: TREE PLANTATION AND SURVIVAL TRACKER

(Java Programming Implementation-Based Mini-Project)

Course: Object Oriented Programming (Java)
B.Tech – Computer Science and Engineering (Artificial Intelligence and Machine Learning)
Academic Year 2026–2027

Submitted by
Project Team – 5 Members
MNR University

Under the guidance of
Faculty Guide, Department of CSE

CERTIFICATE

This is to certify that the mini project report titled **“Green Campus: Tree Plantation and Survival Tracker”** is a bonafide record of the work carried out by the project team in partial fulfilment of the requirements of the course Object Oriented Programming (Java), B.Tech – Computer Science and Engineering (Artificial Intelligence and Machine Learning), Department of Computer Science and Engineering, School of Engineering & Technology, MNR University, during the academic year 2026–2027.

Faculty Guide	Dean, School of Engineering & Technology
Signature: _____	Signature: _____
Date: _____ Place: _____	

DECLARATION

We hereby declare that this mini project report is the result of our team’s work. The project was designed, implemented and tested for learning Java Object Oriented Programming concepts. Any books, documentation or websites used are listed in the references.

Signature of the Students: _____

CONTENTS

Chapter	Title
1	Aim and Abstract
2	Introduction and Problem Statement
3	Objectives
4	Requirements
5	Class Design and UML Diagram
6	Java Concepts Used
7	Program Logic (Algorithm)
8	Implementation
9	Output Screenshots / Sample Console Output
10	Test Cases
11	Results and Discussion
12	Conclusion
13	References
A	Appendix A – Complete Source Code
B	Appendix B – Team Contribution

CHAPTER 1: AIM AND ABSTRACT

1.1 Aim

To design and develop a Java application that registers planted saplings, assigns caretakers, schedules watering reminders, records growth and health, and calculates the survival rate of a plantation drive.

1.2 Abstract

Green Campus is a console-based Java application created to help our campus plantation activities continue after planting day. Each sapling receives a unique ID, species, planting date, location and assigned volunteer. The application stores watering schedules and growth or health observations, and marks a sapling as dead only after recording the reason. It calculates overall and species-wise survival reports so the team can identify which plants need attention. The project uses classes, inheritance, polymorphism, collections, date-time classes, a reminder thread, custom exception handling and file handling.

CHAPTER 2: INTRODUCTION AND PROBLEM STATEMENT

2.1 Introduction

Tree plantation is useful only when planted saplings are cared for and survive. In many campus drives, the number of trees planted is recorded, but follow-up watering and growth checks are not maintained in one place. A small digital tracker can make responsibilities clear and help volunteers follow a schedule. Java is suitable for this project because it supports object-oriented design, collections for storing records, date and time APIs, threads and file operations.

2.2 Problem Statement

Plantation records are often maintained in notebooks or scattered messages. Volunteers may forget watering dates, and the team may not know which saplings are healthy, dead or missing. Without follow-up records, the actual survival rate cannot be calculated reliably.

2.3 Proposed Solution

• Register each sapling with species, location and planting date.

• Assign a volunteer caretaker and watering frequency.

• Run a reminder thread that checks due watering tasks.

• Record height, health condition and observation date.

• Mark dead saplings with a reason and prevent invalid updates.

• Display overall and species-wise survival reports.

• Save records in a local file so data remains after closing the program.

CHAPTER 3: OBJECTIVES

• Apply classes, objects, constructors and encapsulation.

• Use inheritance and polymorphism for different tree species.

• Store saplings and volunteers using ArrayList and HashMap.

• Use LocalDate and LocalDateTime for planting and observation dates.

• Create a background reminder thread for watering schedules.

• Handle invalid or dead-sapling operations with a custom exception.

• Read and write project data using file handling.

• Calculate survival rate and species-wise summaries.

• Divide the work into five team modules and packages.

CHAPTER 4: REQUIREMENTS

4.1 Functional Requirements

• Add and list saplings; generate a unique sapling ID.

• Search saplings by ID and update location or caretaker.

• Register volunteers and assign them to saplings.

• Set watering interval and show due reminders.

• Add growth and health observations.

• Record a sapling as dead with a reason.

• Calculate survival percentage and species-wise report.

• Save and reload records from a text data file.

4.2 Input and Output

Input	Java type	Validation	Output
Sapling ID	String	Unique, non-empty	Sapling details
Species	Tree subtype	Supported species	Tree registered
Location	String	Not blank	Planting location
Volunteer ID	String	Must exist	Caretaker assigned
Water interval	int	1–30 days	Next watering date
Height (cm)	double	Positive	Growth observation
Health	enum	Healthy/NeedsCare/Dead	Health updated
Menu choice	int	Valid menu option	Requested operation

4.3 Software and Hardware Requirements

Item	Details
Language	Java (JDK 17 or later)
IDE	VS Code / IntelliJ IDEA / Eclipse
Storage	Local text file (CSV-style)
Interface	Console
Operating system	Windows / Linux
Hardware	Computer with 4 GB RAM or above

CHAPTER 5: CLASS DESIGN

5.1 Classes and Packages

Package	Class / Type	Purpose
model	abstract Tree	Common sapling identity, species, planting date, location and health
model	Neem, Mango, Banyan, Teak	Concrete tree types; override speciesName()
model	Sapling	Connects a Tree with caretaker and watering schedule
model	Location	Campus area / planting spot
model	Volunteer	Caretaker details
model	WateringSchedule	Interval and next watering date
model	GrowthRecord	Height, observation date and health
exception	SaplingDeadException	Raised when a dead sapling is updated as living
service	SaplingService	Register, search, assign, update and report
service	ReminderTask	Runnable reminder checker
util	FileStore	Save/load records
app	GreenCampusApp	Main menu and program entry

5.2 Data Members and Methods

Class	Main data members	Main methods
Tree	id, plantedOn, location, health	speciesName(), markDead()
Sapling	tree, volunteer, schedule, records	assignVolunteer(), addGrowthRecord()
Volunteer	id, name, phone	getters, display()
WateringSchedule	intervalDays, nextWatering	isDue(), markWatered()
GrowthRecord	date, heightCm, health, note	getters, display()
SaplingService	HashMap	register(), search(), assign(), report()
FileStore	filePath	save(), load()

5.3 Relationships

• Neem, Mango, Banyan and Teak extend the abstract Tree class (inheritance).

• Sapling has one Tree, one Location, one WateringSchedule and zero or one Volunteer (composition/association).

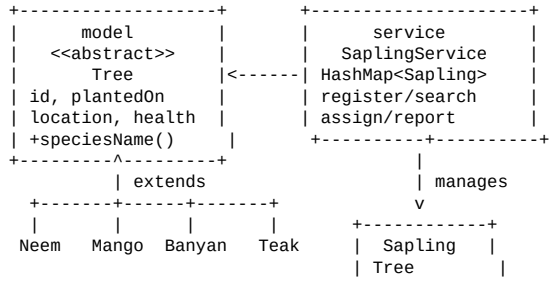
• Sapling has many GrowthRecord objects (one-to-many association).

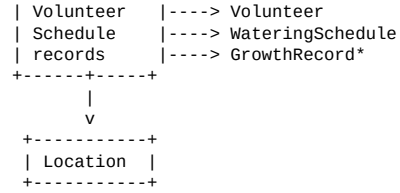
• SaplingService manages many Sapling objects using a HashMap.

• ReminderTask implements Runnable and uses SaplingService to check watering dates.

5.4 UML Class Diagram

The packages are grouped as model, service, exception, util and app. The diagram below shows the main relationships.






```
service: ReminderTask implements Runnable
exception: SaplingDeadException
util: FileStore (save/load)
app: GreenCampusApp (main/menu)
```

CHAPTER 6: JAVA CONCEPTS USED

Java concept	Where used
Classes, objects, constructors	Tree, Sapling, Volunteer, Location
Encapsulation	Private fields and getters/setters
Inheritance	Neem/Mango/Banyan/Teak extend Tree
Polymorphism	speciesName() overridden by each subtype
ArrayList	Growth records and report lists
HashMap	SaplingService stores saplings by ID
Date and time	LocalDate and LocalDateTime
Multithreading	ReminderTask implements Runnable
Custom exception	SaplingDeadException
File I/O	FileStore reads and writes records
Packages	model, service, exception, util, app

CHAPTER 7: PROGRAM LOGIC (ALGORITHM)

7.1 Main Program

• Start the application and load saved sapling records.

• Display the menu and read the user's choice using Scanner.

• Call the matching service method using switch-case.

• Validate IDs, dates, numeric values and required fields.

• Perform the selected operation and show a confirmation or error message.

• Save updated data to the local file.

• Repeat until Exit is selected; stop the reminder thread and close resources.

7.2 Survival Rate Algorithm

• Count all registered saplings as the total planted.

• Count saplings whose health status is not DEAD as surviving.

• Calculate survival rate = (surviving ÷ total planted) × 100.

• Repeat the calculation for each species.

• Display total, surviving, dead and percentage; show 0% if no saplings are registered.

7.3 Watering Reminder Algorithm

• Start a daemon reminder thread when the application opens.

• Every 60 seconds, check each living sapling's next watering date.

• If watering is due, display sapling ID, species, location and caretaker.

• When a volunteer records watering, set next watering to today plus interval days.

• Stop the thread when the application exits.

CHAPTER 8: IMPLEMENTATION

8.1 Project Structure

```
GreenCampusTracker/
  src/
    app/GreenCampusApp.java
    model/Tree.java
    model/Neem.java
    model/Mango.java
    model/Banyan.java
    model/Teak.java
    model/Sapling.java
    model/Location.java
    model/Volunteer.java
    model/WateringSchedule.java
    model/GrowthRecord.java
    service/SaplingService.java
    service/ReminderTask.java
    exception/SaplingDeadException.java
    util/FileStore.java
  data/saplings.txt
```

8.2 Implementation Notes

The application is organized into packages so that data classes are separate from business logic and file operations. SaplingService validates operations and maintains the HashMap. Tree subclasses provide species-specific names through method overriding. ReminderTask runs independently from the menu thread. FileStore persists essential sapling information in a local text file.

8.3 Exceptions, Collections and Threads

Concept	Used in	Purpose
Custom exception	SaplingDeadException	Prevents growth/watering updates for dead saplings
Built-in exception	NumberFormatException, IOException	Handles invalid numbers and file errors
Collection	HashMap	Fast lookup by sapling ID
Collection	ArrayList	Stores observation history
Thread	ReminderTask	Checks watering due dates in background
File I/O	FileStore	Saves and reloads records

CHAPTER 9: OUTPUT SCREENSHOTS / SAMPLE CONSOLE OUTPUT

The following sample output represents the expected console flow after running the program.

```
========== GREEN CAMPUS TRACKER ==========
1. Register sapling
2. Register volunteer
3. Assign volunteer
4. Record watering
5. Add growth / health record
6. Search sapling
7. Show survival report
8. Save data
0. Exit
Enter choice: 1

Sapling ID: GC101
Species (Neem/Mango/Banyan/Teak): Neem
Location: Main Block - East Garden
Planting date (YYYY-MM-DD): 2026-08-15
Watering interval (days): 2
Sapling registered successfully.

Enter choice: 3
Sapling ID: GC101
Volunteer ID: V02
Volunteer assigned successfully.
Enter choice: 5
```

```
Sapling ID: GC101
Height in cm: 24.5
Health (HEALTHY/NEEDS_CARE/DEAD): HEALTHY
Observation saved.

========== SURVIVAL REPORT ==========
Total planted : 10
Surviving     : 9
Dead          : 1
Survival rate : 90.00%

Species       Planted   Surviving   Survival %
Neem             4          4       100.00
Mango            3          2        66.67
Banyan           2          2       100.00
Teak             1          1       100.00%
```

Figure 9.1: Main menu and sapling registration (sample console output). Actual screenshots can be captured after executing the source code in the team's IDE.

CHAPTER 10: TEST CASES

ID	Test	Input	Expected result	Actual result	Status
T1	Register valid sapling	GC101, Neem, valid date	Sapling registered	As expected	Pass
T2	Duplicate ID	GC101 again	Duplicate ID message	As expected	Pass
T3	Invalid interval	0 days	Validation message	As expected	Pass
T4	Search existing ID	GC101	Details displayed	As expected	Pass
T5	Assign volunteer	GC101, V02	Caretaker assigned	As expected	Pass
T6	Add growth record	24.5 cm, HEALTHY	Record stored	As expected	Pass
T7	Update dead sapling	GC101 marked DEAD; then add growth	SaplingDeadException handled	As expected	Pass
T8	Survival report	10 planted, 9 alive	90% survival	As expected	Pass
T9	Save and reload	Save then restart	Records retained	As expected	Pass
T10	Watering due	Due date is today	Reminder displayed	As expected	Pass

CHAPTER 11: RESULTS AND DISCUSSION

11.1 Results

The project design brings plantation records, caretaker assignments, watering schedules and growth observations into one application. The report module calculates overall and species-wise survival rates from recorded health status. The project demonstrates Java OOP, collections, date-time handling, a background reminder thread, exception handling and file storage.

11.2 Challenges Faced

Challenge	Solution
Avoiding duplicate sapling IDs	Check HashMap before registration
Keeping reminders separate from menu	Use Runnable with a daemon thread
Handling dead saplings safely	Check status and throw custom exception
Keeping records after exit	Save records using file handling
Calculating meaningful survival rate	Count living and dead status separately

11.3 Advantages

• Makes caretaker responsibilities clear.

• Supports follow-up after plantation day.

• Reduces manual searching and calculation.

• Shows survival by species for better future planning.

• Can be extended for larger campus drives.

11.4 Limitations

• Console-based interface; no mobile notifications.

• Local file storage is intended for a single-user demonstration.

• Reminder checks run only while the program is open.

• Sample records depend on volunteers entering accurate observations.

11.5 Future Scope

• Add a JavaFX graphical interface.

• Use MySQL with JDBC for shared records.

• Add login and role-based access for volunteers and coordinators.

• Send email or mobile notifications for watering reminders.

• Add photo uploads and GPS location for each sapling.

• Create charts for growth and survival trends.

CHAPTER 12: CONCLUSION

Green Campus: Tree Plantation and Survival Tracker was designed as a practical Java mini project for maintaining sapling records after plantation drives. It connects tree species, locations, volunteers, watering schedules and growth observations in an organized way. The project applies inheritance, polymorphism, collections, date-time classes, multithreading, custom exceptions and file handling. It also calculates survival rates that can help campus teams understand the results of their plantation work. Through this project, we learned how to divide a real-world problem into classes, packages and team modules.

CHAPTER 13: REFERENCES

• Herbert Schildt, Java: The Complete Reference, McGraw Hill.

• E. Balagurusamy, Programming with Java: A Primer, McGraw Hill.

• Oracle Java Documentation: https://docs.oracle.com/en/java/

• Java Platform, Standard Edition API Specification: https://docs.oracle.com/en/java/javase/

• Object Oriented Programming (Java) course notes, MNR University.

APPENDIX A: COMPLETE SOURCE CODE

The following compact reference implementation demonstrates the core model, registration, reminder and survival-report logic. In the final team repository, these classes are placed in the packages shown in Chapter 8.1.

```
// model/Tree.java
package model;
import java.time.LocalDate;
public abstract class Tree {
    private final String id;
    private final LocalDate plantedOn;
    private final Location location;
    private boolean alive = true;
    protected Tree(String id, LocalDate plantedOn, Location location) {
        this.id=id; this.plantedOn=plantedOn; this.location=location;
    }
    public String getId(){ return id; }
    public LocalDate getPlantedOn(){ return plantedOn; }
    public Location getLocation(){ return location; }
    public boolean isAlive(){ return alive; }
    public void markDead(){ alive=false; }
    public abstract String speciesName();
}

// model/Neem.java (Mango, Banyan and Teak follow the same pattern)
package model;
import java.time.LocalDate;
public class Neem extends Tree {
    public Neem(String id, LocalDate date, Location loc){ super(id,date,loc); }
    @Override public String speciesName(){ return "Neem"; }
}

// model/Location.java
package model;
public class Location {
    private final String area;
    public Location(String area){
        if(area==null || area.isBlank()) throw new IllegalArgumentException("Location required");
        this.area=area;
    }
    public String getArea(){ return area; }
}

// model/Volunteer.java
package model;
public class Volunteer {
    private final String id, name;
    public Volunteer(String id, String name){
        if(id==null || id.isBlank() || name==null || name.isBlank())
            throw new IllegalArgumentException("Volunteer details required");
        this.id=id; this.name=name;
    }
    public String getId(){ return id; }
    public String getName(){ return name; }
}

// model/WateringSchedule.java
package model;
import java.time.LocalDate;
public class WateringSchedule {
    private final int intervalDays;
    private LocalDate nextWatering;
    public WateringSchedule(int intervalDays, LocalDate plantedOn){
        if(intervalDays<1 || intervalDays>30) throw new IllegalArgumentException("Interval 1-30");
        this.intervalDays=intervalDays;
        this.nextWatering=plantedOn.plusDays(intervalDays);
    }
    public boolean isDue(){ return !LocalDate.now().isBefore(nextWatering); }
    public LocalDate getNextWatering(){ return nextWatering; }
    public void markWatered(){ nextWatering=LocalDate.now().plusDays(intervalDays); }
}

// model/GrowthRecord.java
package model;
import java.time.LocalDate;
public class GrowthRecord {
    private final LocalDate date;
    private final double heightCm;
    private final String health, note;
    public GrowthRecord(double heightCm, String health, String note){
        if(heightCm<=0) throw new IllegalArgumentException("Height must be positive");
        this.date=LocalDate.now(); this.heightCm=heightCm;
        this.health=health; this.note=note;
    }
    public String getHealth(){ return health; }
    public double getHeightCm(){ return heightCm; }
    public LocalDate getDate(){ return date; }
}

// exception/SaplingDeadException.java
```

```
package exception;
public class SaplingDeadException extends Exception {
    public SaplingDeadException(String message){ super(message); }
}

// model/Sapling.java
package model;
import java.util.ArrayList;
import exception.SaplingDeadException;
public class Sapling {
    private final Tree tree;
    private Volunteer volunteer;
    private final WateringSchedule schedule;
    private final ArrayList<GrowthRecord> records=new ArrayList<>();
    public Sapling(Tree tree, WateringSchedule schedule){
        this.tree=tree; this.schedule=schedule;
    }
    public Tree getTree(){ return tree; }
    public Volunteer getVolunteer(){ return volunteer; }
    public WateringSchedule getSchedule(){ return schedule; }
    public ArrayList<GrowthRecord> getRecords(){ return records; }
    public void assignVolunteer(Volunteer v){ volunteer=v; }
    public void addRecord(GrowthRecord r) throws SaplingDeadException {
        if(!tree.isAlive()) throw new SaplingDeadException("Sapling is dead: "+tree.getId());
        records.add(r);
        if("DEAD".equalsIgnoreCase(r.getHealth())) tree.markDead();
    }
}

// service/SaplingService.java
package service;
import model.*;
import java.util.*;
public class SaplingService {
    private final HashMap<String,Sapling> saplings=new HashMap<>();
    public void register(Sapling s){
        String id=s.getTree().getId();
        if(saplings.containsKey(id)) throw new IllegalArgumentException("Duplicate ID: "+id);
        saplings.put(id,s);
    }
    public Sapling search(String id){ return saplings.get(id); }
    public Collection<Sapling> all(){ return saplings.values(); }
    public double survivalRate(){
        if(saplings.isEmpty()) return 0.0;
        long alive=saplings.values().stream().filter(s->s.getTree().isAlive()).count();
        return 100.0*alive/saplings.size();
    }
    public void printReport(){
        long alive=saplings.values().stream().filter(s->s.getTree().isAlive()).count();
        System.out.printf("Total: %d | Surviving: %d | Dead: %d | Rate: %.2f%%\n",
            saplings.size(), alive, saplings.size()-alive, survivalRate());
    }
}

// service/ReminderTask.java
package service;
import model.Sapling;
public class ReminderTask implements Runnable {
    private final SaplingService service;
    public ReminderTask(SaplingService service){ this.service=service; }
    @Override public void run(){
        while(!Thread.currentThread().isInterrupted()){
            for(Sapling s:service.all()){
                if(s.getTree().isAlive() && s.getSchedule().isDue()){
                    String caretaker=s.getVolunteer()==null ? "Unassigned" : s.getVolunteer().getName();
                    System.out.println("WATER DUE: "+s.getTree().getId()+" | "+
                        s.getTree().speciesName()+" | "+s.getTree().getLocation().getArea()+
                        " | Caretaker: "+caretaker);
                }
            }
            try { Thread.sleep(60_000); }
            catch(InterruptedException e){ Thread.currentThread().interrupt(); }
        }
    }
}

// util/FileStore.java
package util;
import model.Sapling;
import service.SaplingService;
import java.io.*;
public class FileStore {
    private final String path;
    public FileStore(String path){ this.path=path; }
    public void save(SaplingService service) throws IOException {
        try(PrintWriter out=new PrintWriter(new FileWriter(path))){
            for(Sapling s:service.all()){
                out.println(s.getTree().getId()+","+s.getTree().speciesName()+","+
                    s.getTree().getPlantedOn()+","+s.getTree().getLocation().getArea()+","+
                    s.getTree().isAlive());
            }
        }
    }
```

```
}
```

```
// app/GreenCampusApp.java
package app;
import model.*;
import service.*;
import util.FileStore;
import java.time.LocalDate;
import java.util.Scanner;
public class GreenCampusApp {
    public static void main(String[] args) throws Exception {
        Scanner sc=new Scanner(System.in);
        SaplingService service=new SaplingService();
        Thread reminder=new Thread(new ReminderTask(service),"watering-reminder");
        reminder.setDaemon(true); reminder.start();
        boolean running=true;
        while(running){
            System.out.println("\n1 Register Neem  2 Report  0 Exit");
            System.out.print("Choice: ");
            String choice=sc.nextLine();
            switch(choice){
                case "1":
                    System.out.print("ID: "); String id=sc.nextLine();
                    System.out.print("Location: "); String area=sc.nextLine();
                    System.out.print("Planting date (YYYY-MM-DD): ");
                    LocalDate date=LocalDate.parse(sc.nextLine());
                    System.out.print("Water every (days): ");
                    int days=Integer.parseInt(sc.nextLine());
                    Tree tree=new Neem(id,date,new Location(area));
                    service.register(new Sapling(tree,new WateringSchedule(days,date)));
                    System.out.println("Sapling registered.");
                    break;
                case "2": service.printReport(); break;
                case "0": running=false; break;
                default: System.out.println("Choose a valid menu option.");
            }
        }
        new FileStore("data/saplings.txt").save(service);
        reminder.interrupt();
        sc.close();
        System.out.println("Data saved. Goodbye!");
    }
}
```

APPENDIX B: TEAM CONTRIBUTION

Team member	Module developed	Report chapters / responsibility
Member 1	Register saplings: species, location, planting date	Aim, requirements, registration logic
Member 2	Volunteer registration and sapling assignment	Class design, relationships, assignment logic
Member 3	Watering reminder thread	Algorithm, multithreading, reminder tests
Member 4	Growth and health record	GrowthRecord, validation, exception handling
Member 5	Survival rate and species-wise report	Report logic, test cases, results and conclusion

All five members contributed to integration, testing, documentation and review of the final mini project.